

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

ЗВІТ
з лабораторної роботи № 4
з дисципліни «Основи програмування»
«Основні типи змінних. Операції та їх пріоритет»

Виконала:
студент групи ФЕП-13
Савчак Дарина
Перевірив:
асистент Васюта В. М.

Львів 2026

Мета роботи:

Вивчити поняття: основні типи змінних, вказівники, операції та їх пріоритет.

Обладнання та програмне забезпечення:

- IBM сумісна персональна обчислювальна машина;
- онлайн компілятор мови програмування Cі, доступний за посиланням <https://repl.it/languages/c>

Завдання до роботи:

Написати програму для з використанням мови програмування Cі, котра реалізує наступні функції:

- використання основних типів змінних;
- реалізацію всіх операторів;
- обчислення складних математичних виразів.

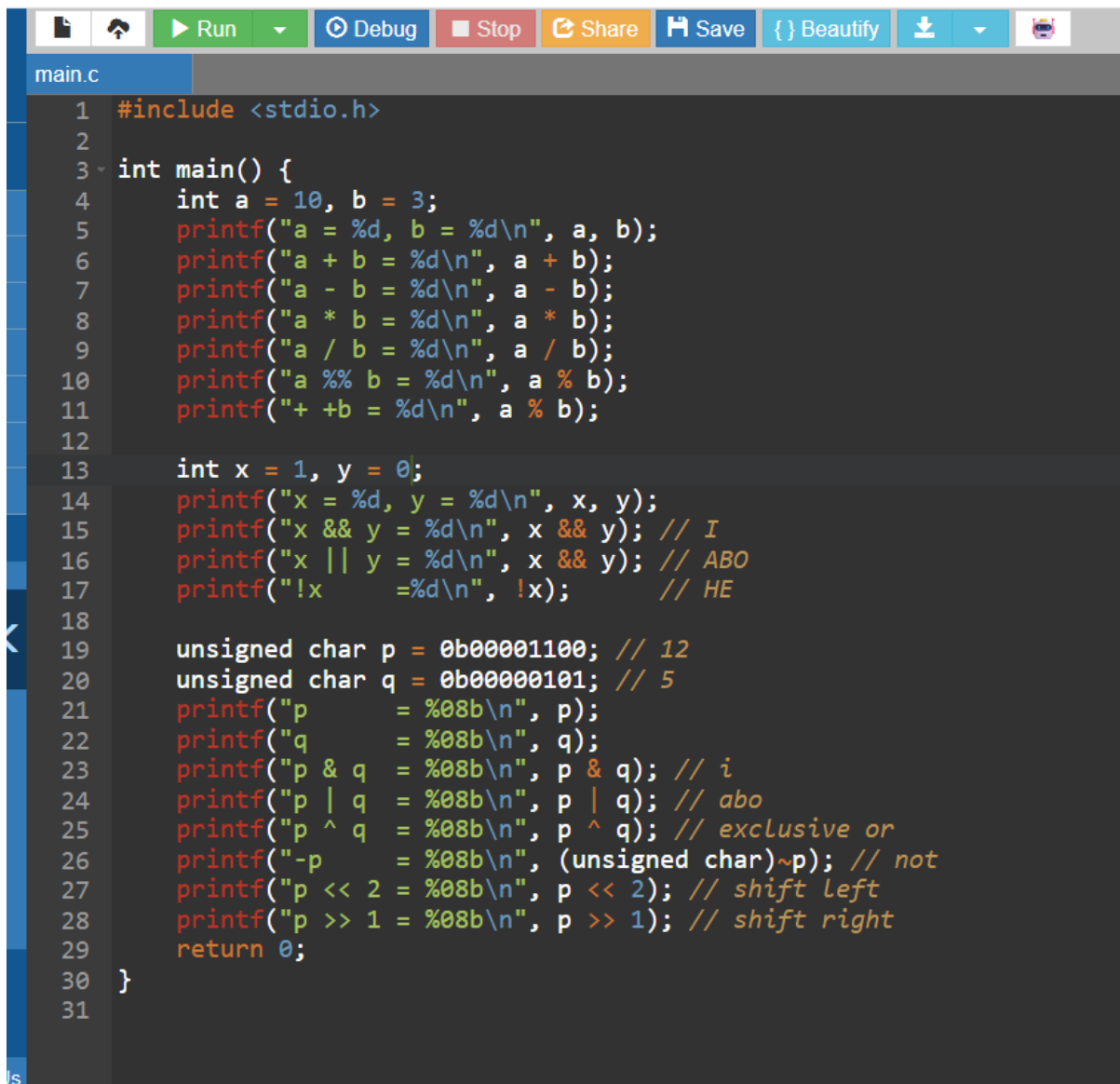
Теоретичні відомості

1. Базові типи даних у Cі: char (символьний); цілочисельні (short, int, long, long long та їх беззнакові модифікатори unsigned); дійсні з одинарною (float) та подвійною (double) точністю; void (пустий тип — для функцій без значень/параметрів та нетипізованих вказівників).
2. Класи операторів та пріоритет: Арифметичні: +, -, *, /, %, ++, --. Порівняння: >, >=, <, <=, ==, !=. Логічні: && (І), || (АБО), ! (НЕ). Побітові: &, |, ^, ~, >>, <<. Пріоритет: найвищий у унарних операторів, *, /, % та круглих дужок (); найнижчий — у бінарних +, -. Оператори одного пріоритету виконуються зліва направо.
3. Вказівники: Вказівник — це змінна, яка зберігає адресу іншої змінної в пам'яті. Операція взяття адреси (&) повертає адресу змінної. Операція розіменування (*) повертає значення за вказаною адресою.
4. Математична бібліотека math.h: Надає функції для математичних обчислень (sqrt, pow, sin, cos, log, fabs тощо). Усі функції, що працюють із кутами, приймають значення виключно в радіанах.

Хід роботи

Завдання 1. Арифметичні, логічні та побітові оператори

Програма демонструє арифметику для $a = 10$, $b = 3$, логічні операції для $x = 1$, $y = 0$ та побітові операції над `unsigned char` у двійковому вигляді (`%08b`).



```
main.c
1  #include <stdio.h>
2
3  int main() {
4      int a = 10, b = 3;
5      printf("a = %d, b = %d\n", a, b);
6      printf("a + b = %d\n", a + b);
7      printf("a - b = %d\n", a - b);
8      printf("a * b = %d\n", a * b);
9      printf("a / b = %d\n", a / b);
10     printf("a %% b = %d\n", a % b);
11     printf("+ +b = %d\n", a % b);
12
13     int x = 1, y = 0;
14     printf("x = %d, y = %d\n", x, y);
15     printf("x && y = %d\n", x && y); // I
16     printf("x || y = %d\n", x || y); // ABO
17     printf("!x      = %d\n", !x);    // HE
18
19     unsigned char p = 0b00001100; // 12
20     unsigned char q = 0b0000101;  // 5
21     printf("p      = %08b\n", p);
22     printf("q      = %08b\n", q);
23     printf("p & q   = %08b\n", p & q); // i
24     printf("p | q   = %08b\n", p | q); // abo
25     printf("p ^ q   = %08b\n", p ^ q); // exclusive or
26     printf("-p      = %08b\n", (unsigned char)~p); // not
27     printf("p << 2 = %08b\n", p << 2); // shift left
28     printf("p >> 1 = %08b\n", p >> 1); // shift right
29     return 0;
30 }
31
```

Рис. 1 код програми (task.1.c)

Результат виконання

```
a = 10, b = 3
a + b = 13
a - b = 7
a * b = 30
a / b = 3
a % b = 1
++b = 1
x = 1, y = 0
x && y = 0
x || y = 0
!x = 0
p = 00001100
q = 00000101
p & q = 00000100
p | q = 00001101
p ^ q = 00001001
~p = 11110011
p << 2 = 00110000
p >> 1 = 00000110

...Program finished with exit code 0
Press ENTER to exit console.
```

Арифметичний блок: під час обчислень із числами 10 та 3 сума становить 13, різниця — 7, множення дає 30, цілочисельне ділення дорівнює 3, а залишок від ділення складає 1. Застосування оператора інкремента (++b) збільшує змінну до 4.

Логічні вирази: для значень 1 та 0 операція кон'юнкції (&&) повертає 0, диз'юнкція (||) дає 1, а логічне заперечення (!) для одиниці дорівнює 0.

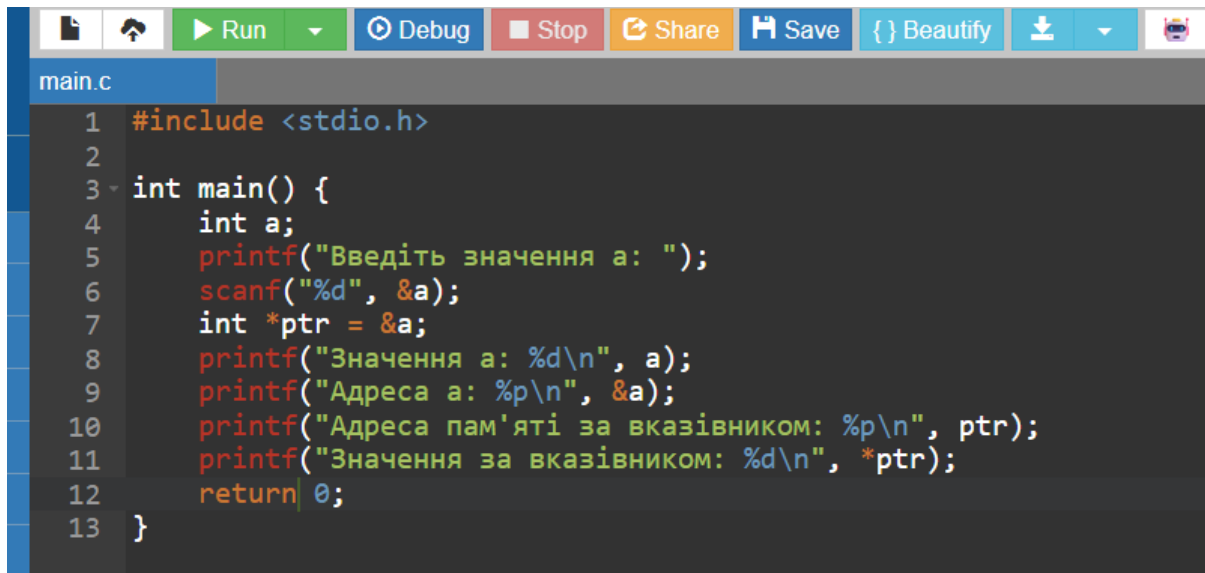
Побітові операції: для двійкових змінних $p = 00001100$ та $q = 00000101$ побітове «І» формує 00000100, «АБО» — 00001101, виключне «АБО» — 00001001, інверсія $\sim p$ дає 11110011, а операції зсуву вліво ($<<2$) та вправо ($>>1$) коректно зсувають розряди.

Завдання 2. Дослідження роботи з вказівниками та змінними

У середовищі OnlineGDB було реалізовано програму на мові C, яка демонструє роботу зі звичайними змінними та вказівниками в пам'яті комп'ютера.

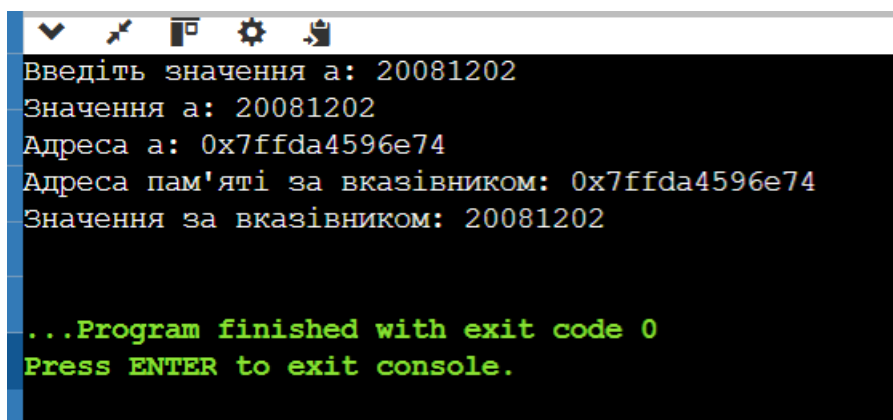
Спочатку оголошується цілочисельна змінна `int a`; За допомогою функцій `printf` та `scanf` користувачеві пропонується ввести з клавіатури довільне

ціле число (у нашому прикладі було введено значення 20081202). Далі створюється вказівник `int *ptr = &a;`, який зберігає адресу цієї змінної в пам'яті. Програма виводить на екран чотири параметри: саму введену змінну, її адресу в пам'яті (`&a`), адресу, на яку посилається вказівник (`ptr`), та значення, яке отримується за вказівником (`*ptr`).



```
1 #include <stdio.h>
2
3 int main() {
4     int a;
5     printf("Введіть значення a: ");
6     scanf("%d", &a);
7     int *ptr = &a;
8     printf("Значення a: %d\n", a);
9     printf("Адреса a: %p\n", &a);
10    printf("Адреса пам'яті за вказівником: %p\n", ptr);
11    printf("Значення за вказівником: %d\n", *ptr);
12    return 0;
13 }
```

Результат виконання



```
Введіть значення a: 20081202
Значення a: 20081202
Адреса a: 0x7ffda4596e74
Адреса пам'яті за вказівником: 0x7ffda4596e74
Значення за вказівником: 20081202

...Program finished with exit code 0
Press ENTER to exit console.
```

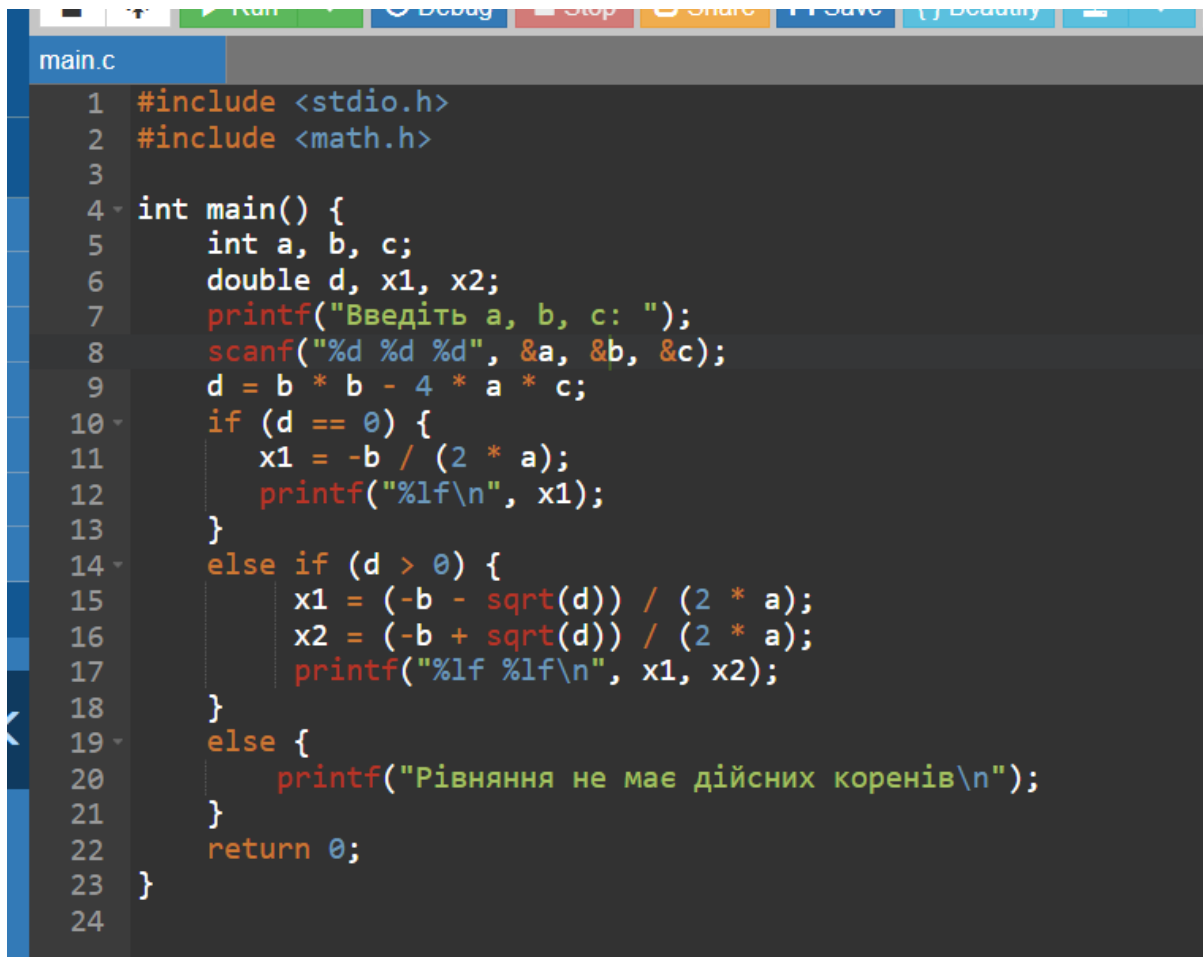
Рис.3. Вивід (task2) (введено 20081202)

Як видно з результатів виконання програми, введене число успішно зчитане та збережене. Адреса змінної (0x7ffda4596e74) повністю збігається з адресою пам'яті, на яку вказує `ptr`, а розіменування вказівника (`*ptr`) коректно повертає початкове значення змінної `a` (20081202). Це доводить правильність роботи механізму прякреференсу та розіменування в пам'яті мови C.

Завдання 3. Розв'язання квадратного рівняння

Для перевірки роботи програми, що реалізує розв'язання квадратного рівняння вигляду $ax^2 + bx + c = 0$, з клавіатури було введено набір коефіцієнтів: $a = 28$, $b = 44$, $c = 18$.

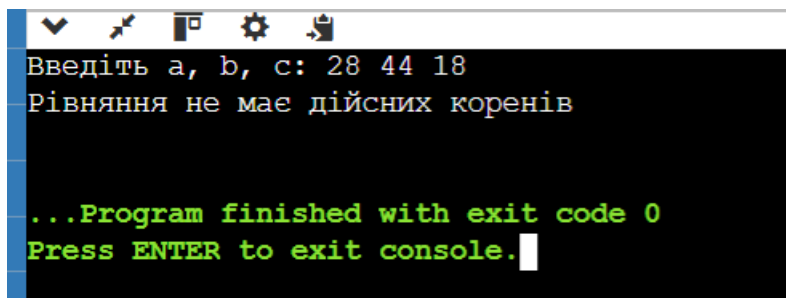
Програма зчитує змінні a , b та c за допомогою функції `scanf("%d %d %d", &a, &b, &c)`; Обчислюється дискримінант за стандартною формулою: $d = b^2 - 4ac = 44^2 - 4 \cdot 28 \cdot 18 = 1936 - 2016 = -80$



```
main.c
1 #include <stdio.h>
2 #include <math.h>
3
4 int main() {
5     int a, b, c;
6     double d, x1, x2;
7     printf("Введіть a, b, c: ");
8     scanf("%d %d %d", &a, &b, &c);
9     d = b * b - 4 * a * c;
10    if (d == 0) {
11        x1 = -b / (2 * a);
12        printf("%lf\n", x1);
13    }
14    else if (d > 0) {
15        x1 = (-b - sqrt(d)) / (2 * a);
16        x2 = (-b + sqrt(d)) / (2 * a);
17        printf("%lf %lf\n", x1, x2);
18    }
19    else {
20        printf("Рівняння не має дійсних коренів\n");
21    }
22    return 0;
23 }
24
```

За допомогою ущільненого розгалуження (if-else) програма аналізує знак дискримінанта: оскільки $d < 0$ (-80), умова додатного чи нульового дискримінанта не виконується. Спрацьовує гілка else, яка виводить на екран інформативне повідомлення: «Рівняння не має дійсних коренів».

Результат виконання



```
Введіть a, b, c: 28 44 18
Рівняння не має дійсних коренів

...Program finished with exit code 0
Press ENTER to exit console.
```

Рис.5. Вивід (task3) (a=28; b=44; c=18)

Висновок

Під час виконання лабораторної роботи №4 було закріплено навички роботи з базовими типами даних, операторами вводу/виводу (printf, scanf) та вказівниками у мові програмування C. На практиці досліджено механізм роботи вказівників: за допомогою операторів взяття адреси (&) та розіменування (*) успішно отримано доступ до комірок пам'яті комп'ютера та перевірено ідентичність адрес змінної та її вказівника. Реалізовано програму для знаходження коренів квадратного рівняння з використанням умовних операторів (if-else) та математичних функцій бібліотеки <math.h>. Продемонстровано коректну обробку розгалуженого обчислення залежно від знака дискримінанта. Наочно переконано в особливості мови C щодо відсутності суворого контролю типів даних, що вимагає пильної уваги під час підбору специфікаторів форматування для запобігання логічним помилкам.

